# 2 Process Management

# Process Management and Scheduling

## LEARNING OUTCOMES

By the end of this module, students will be able to:

1. Explain what is States, Control Blocks, Context Switching
2. Identify the Process Scheduling: Algorithms (FCFS, SJF, RR, Priority Scheduling)
3. Concurrency Issues: Deadlocks, Starvation, Synchronization

Table of Contents

**2.1 PROCESS MANAGEMENT & SCHEDULLING....4**
What is a Process?....4
Process States....5
Process Scheduling....8
Types of Schedulers....8
Scheduling Algorithm....8
Scheduling Queues....9
Schedulers....11
**2.2 TYPES OF SCHEDULLING....14**
First-Come, First-Serve (FCFS)....14
Shortest Job Next (SJN) / Shortest Job First (SJF)....16
Round Robin (RR)....18
Priority Scheduling....20
**2.3 DEADLOCK MANAGEMENT....25**
What is Deadlock?....25
Conditions for Deadlock: (Coffman's Conditions)....26
Deadlock Prevention Techniques:....26
Deadlock Management Example using Resource Allocation Graph (RAG)....29
Summary of Deadlock Handling Approach....29
....30
**2.4 STARVATION MANAGEMENT....30**
Causes of Starvation....31
Effects of Starvation....31
Difference Between Starvation and Deadlock....32
How to Prevent Starvation....32
**2.5 SYNCHRONIZATION IN OS....35**
What is Synchronization?....35
Critical Section Problem....35
Synchronization Mechanisms....36
Synchronization Problems and Solutions....37
Differences Between Deadlocks, Starvation, and Synchronization....37
Conclusion....38
**2.6 SUMMARY....39**
Processes: States, Control Blocks, Context Switching....39
Process Scheduling: Algorithms (FCFS, SJF, RR, Priority Scheduling)....39
Concurrency Issues: Deadlocks, Starvation, Synchronization....39

## 2.1 PROCESS MANAGEMENT & SCHEDULLING

Process Management and Scheduling are critical functions of an operating system (OS) responsible for efficiently managing the execution of multiple tasks (processes) within a computer system. These functions ensure that system resources are allocated effectively, maximizing performance and system stability.

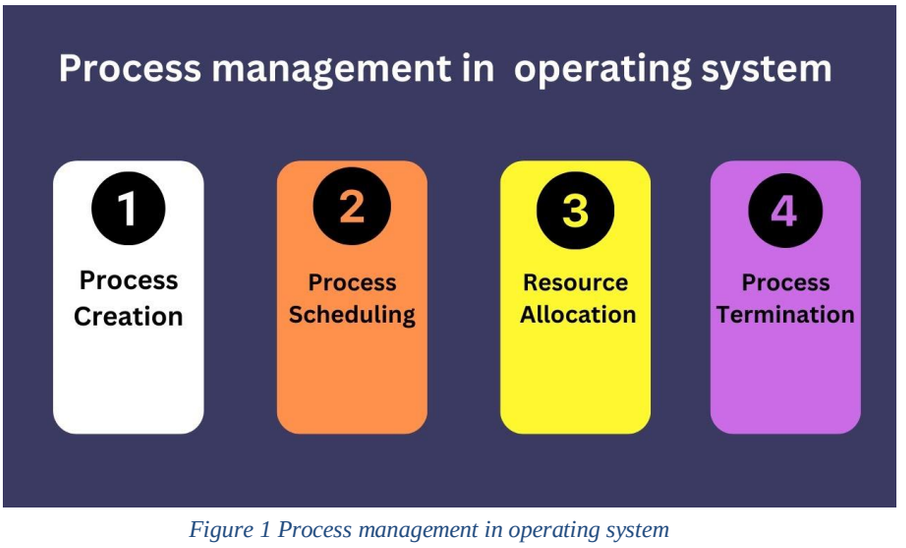



*Figure 1 Process management in operating system*

**What is a Process?**

A process is a running program, not just the program code itself. It includes the current state, such as the program counter and processor registers. Additionally, a process consists of a stack for temporary data like function parameters, return addresses, and local variables, a data section for global variables, and a heap for memory allocated during execution. A program, on the other hand, is just a static set of instructions stored on disk (like an executable file). It becomes a process when loaded into memory and actively executed.

Process Components:

- Program Code: The set of instructions to be executed.
- Process State: Current condition (e.g., ready, running, waiting).
- Program Counter (PC): Indicates the next instruction to be executed.
- Stack: Stores temporary data like method calls and local variables.
- Heap: Dynamically allocated memory used during execution.

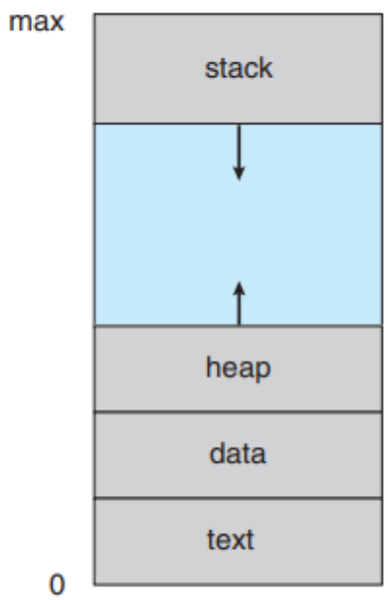



*Figure 2 Process in Memoryy*

## Process States

A process goes through several states during its life cycle:

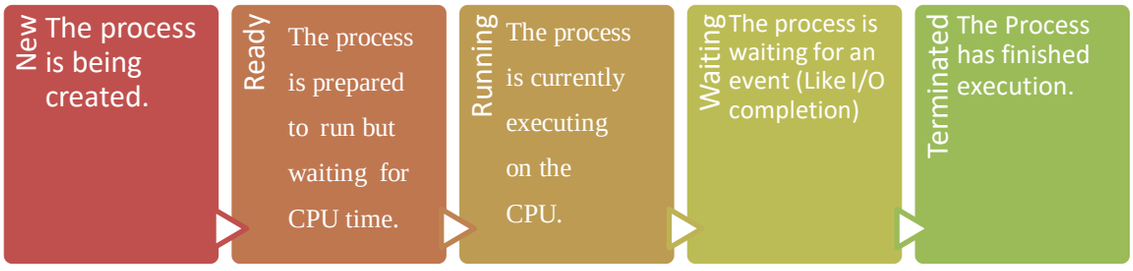



*Figure 3 Process States*

The names for process states may differ across operating systems, but the fundamental states they represent are common to all systems. Some operating systems further break down these states into more specific categories. It's important to note that only one process can actively run on a processor at any given moment, while multiple other processes may be in a ready or waiting state. A state diagram illustrating these process states is shown in Figure 4.

## Process Control Block (PCB)

A Process Control Block (PCB) is a data structure maintained by the OS that contains important information about a process, including:

- Process ID (PID)
- Process State
- Program Counter
- CPU Registers
- Memory Management Information
- I/O Status Information

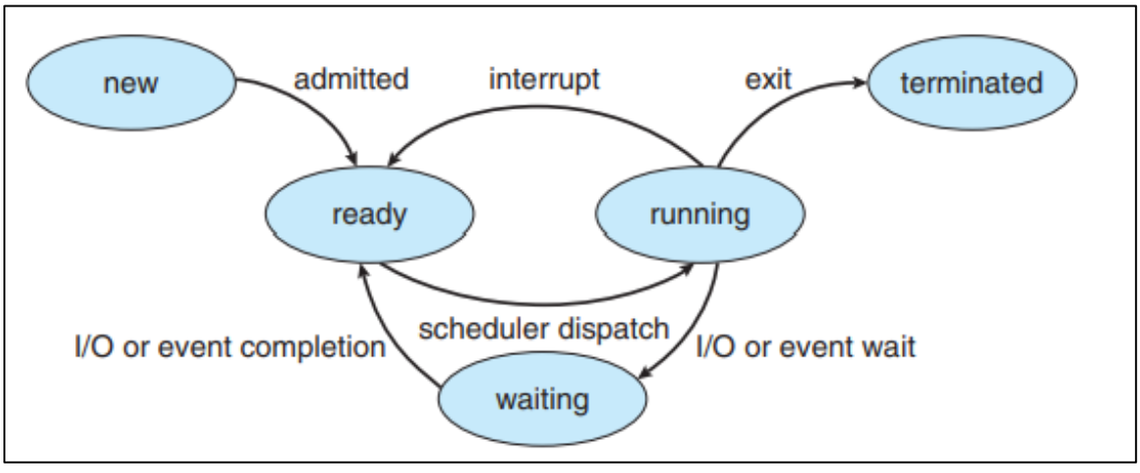



*Figure 4 Diagram of process state.*

- Process State: Describes the current condition of the process, such as new, ready, running, waiting, or halted.
- Program Counter: Holds the address of the next instruction to be executed for the process.
- CPU Registers: These vary based on the computer architecture and include components like accumulators, index registers, stack pointers, general-purpose registers, and condition codes. This information, along with the program counter, must be preserved during an interrupt to ensure the process resumes correctly.
- CPU Scheduling Information: Includes data such as process priority, scheduling queue pointers, and other scheduling-related parameters. (Detailed in Chapter 6 on process scheduling.)
- Memory Management Information: Contains details like base and limit register values, page tables, or segment tables, depending on the system's memory management approach.

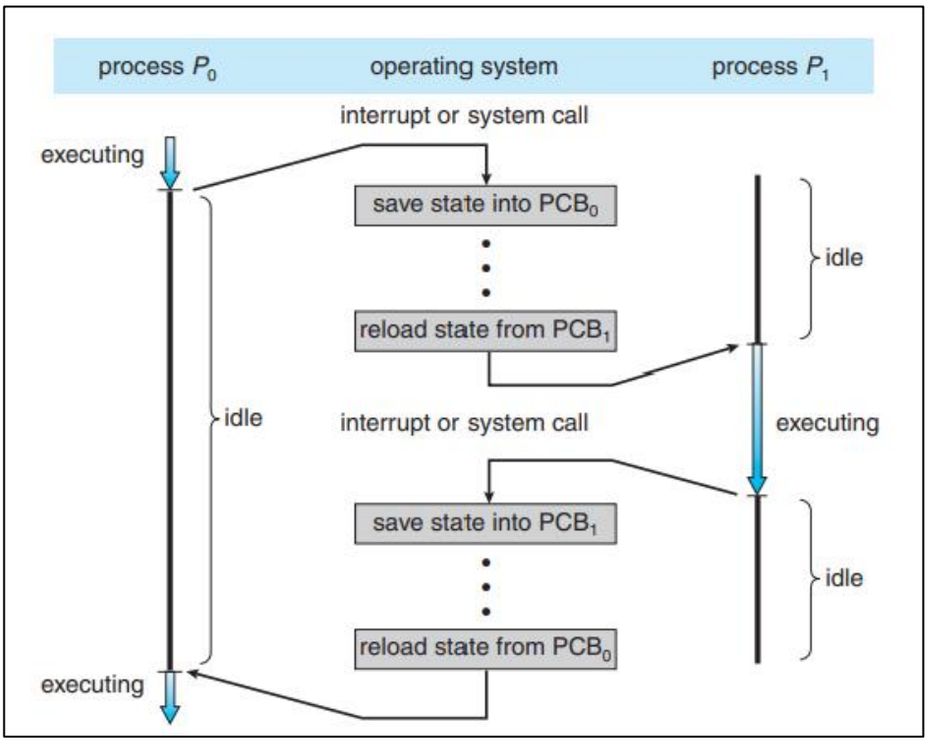



*Figure 5 Diagram showing CPU switch from process to process*

- Accounting Information: Tracks details such as CPU usage, real-time usage, time limits, account numbers, and job or process IDs.
- I/O Status Information: Includes data on the I/O devices assigned to the process and a list of open files.

In summary, the Process Control Block (PCB) acts as a central storage for all information that differs between processes.

**Here are some short-answer questions.**

1. What is a process in an operating system?
2. How does a process differ from a program?
3. List the components of a process in memory.
4. What is the purpose of the program counter in a process?
5. Explain the role of CPU registers in process management.
6. Why is it important to save the program counter and CPU registers during an interrupt?
7. What types of information are included in the memory management section of a PCB?

SEGi University

## Process Scheduling

Process scheduling is a fundamental concept in operating systems that involves the efficient management of multiple processes competing for the CPU and other system resources. It ensures that processes are executed in a fair and efficient manner, maximizing system performance while minimizing waiting time and resource conflicts.

At any given time, an operating system must handle multiple processes, each in different states such as new, ready, running, waiting, or terminated. The process scheduling mechanism determines which process gets to execute on the CPU and for how long. Since a single CPU can execute only one process at a time, process scheduling plays a crucial role in optimizing CPU usage.

### Types of Schedulers

The process scheduling mechanism relies on three types of schedulers:

1. **Long-term Scheduler (Job Scheduler):** Manages the selection of processes to enter the ready queue from the job queue. It controls the degree of multiprogramming.
2. **Short-term Scheduler (CPU Scheduler):** Decides which process from the ready queue gets CPU time next. This decision is critical for CPU efficiency and system responsiveness.
3. **Medium-term Scheduler:** Used in some systems to manage processes that need to be temporarily suspended or swapped out of memory to optimize resource use.

### Scheduling Algorithm

To manage the execution of processes, scheduling algorithms are used. Common scheduling algorithms include:

1. **First-Come, First-Served (FCFS):** Executes processes in the order they arrive, suitable for batch systems but may lead to longer waiting times.
2. **Shortest Job Next (SJN):** Prioritizes the process with the shortest burst time, minimizing waiting time but challenging for real-time systems.
3. **Round Robin (RR):** Assigns a fixed time slice (quantum) to each process, ensuring fairness and responsiveness, commonly used in time-sharing systems.
4. Priority Scheduling: Executes processes based on priority, which can be either preemptive or non-preemptive.
5. **Multilevel Queue Scheduling:** Divides processes into separate queues based on characteristics such as priority or process type.

Process scheduling is essential for balancing several key performance metrics such as CPU utilization, throughput, turnaround time, waiting time, and response time. An effective scheduling system ensures the CPU remains busy while minimizing delays and ensuring fairness among processes.

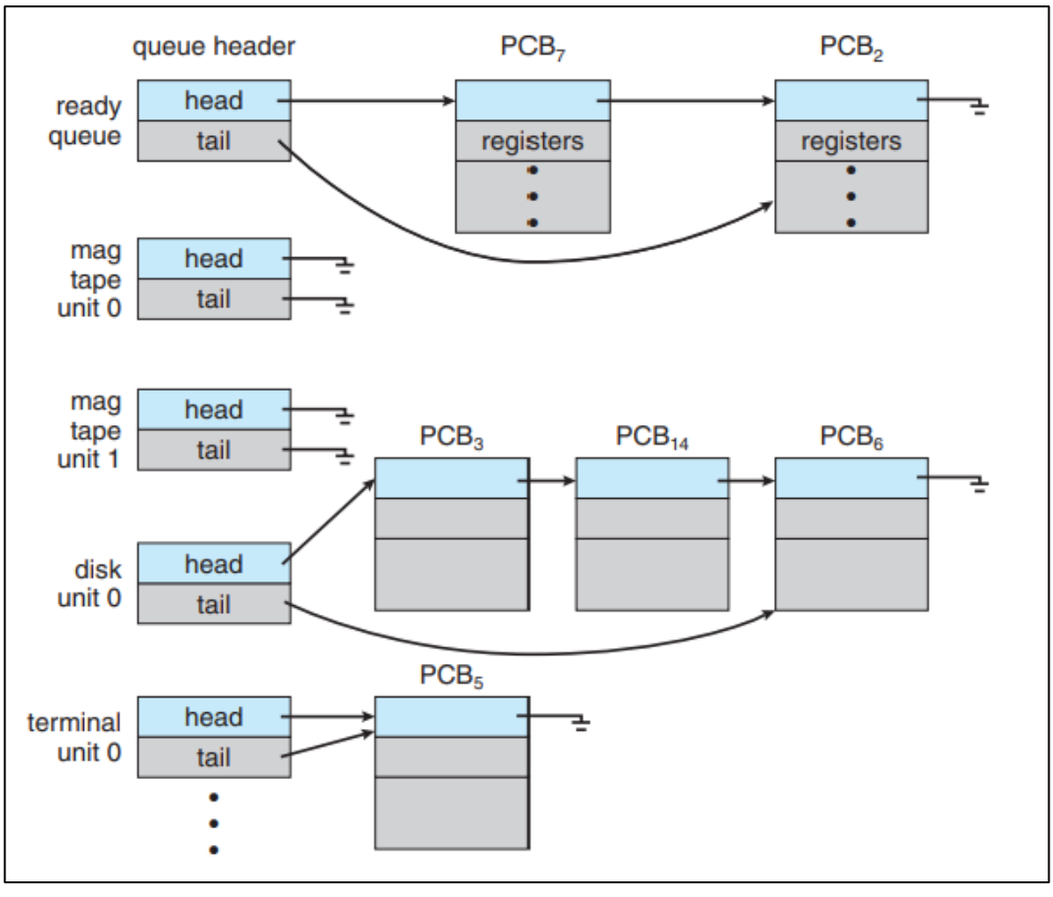



*Figure 6 The ready queue and various I/O device queues.*

In conclusion, process scheduling is vital for maintaining an efficient and responsive computing environment. By carefully managing how processes are selected and executed, the operating system ensures optimal use of resources and improved user experience.

**Scheduling Queues**

When processes enter the system, they are placed in a job queue, which contains all processes in the system. Processes that are loaded into the main memory and ready for execution are stored in the ready queue, typically organized as a linked list. The ready queue header points to the first and last Process Control Blocks (PCBs), with each PCB linking to the next process in the queue.

Other queues also exist in the system. When a process is running on the CPU, it may either finish, be interrupted, or wait for an event like the completion of an I/O operation. If a process requests an I/O device,

such as a disk that is already busy, it will be placed in a device queue specific to that device, waiting for its turn. Each device maintains its own device queue.

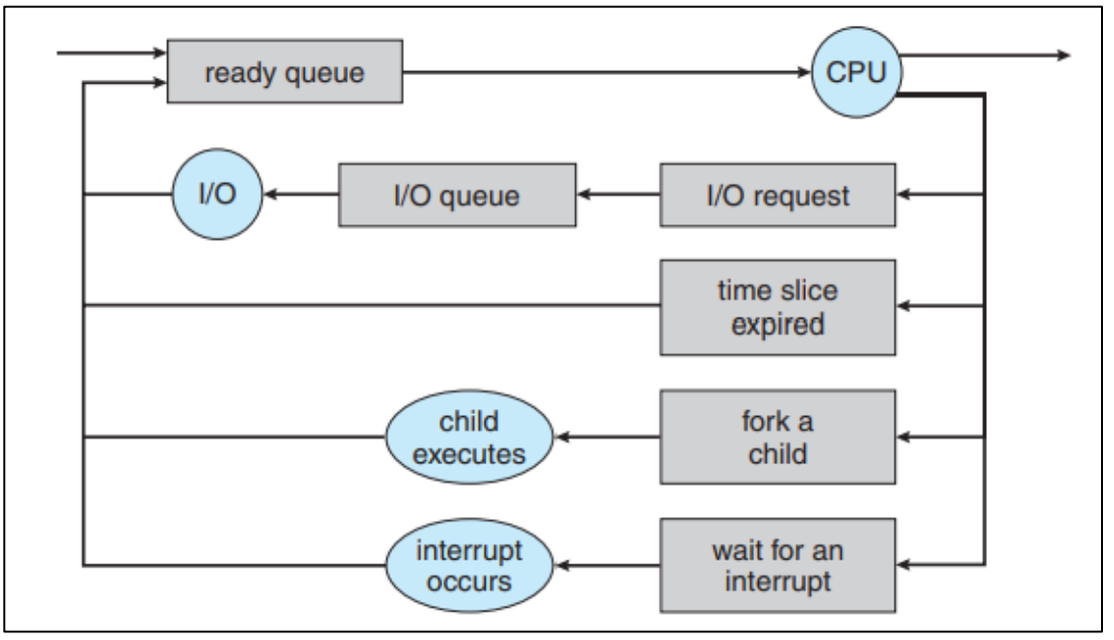



*Figure 7 Queueing diagram representation of process scheduling*

A queueing diagram is commonly used to represent process scheduling. It consists of rectangular boxes representing different queues, such as the ready queue and various device queues. Circles represent system resources handling these queues, while arrows show the movement of processes within the system.

When a process enters the system, it is added to the ready queue and waits until selected for execution (dispatched). Once it begins executing on the CPU, several outcomes can occur:

- The process may request an I/O operation and move to an I/O queue.
- The process may create a child process and wait for its termination.
- The process may be interrupted and returned to the ready queue.
- In the first two cases, the process eventually returns to the ready queue once the waiting event completes. The process repeats this cycle until it terminates, at which point it is removed from all queues, and its PCB and resources are released.

**Schedulers**

A process moves between different scheduling queues throughout its lifecycle. The operating system uses schedulers to select processes for execution from these queues.

1. **Long-Term Scheduler (Job Scheduler):**
   - Selects processes from the job pool (on mass storage) and loads them into memory for execution.
   - Runs less frequently, often when a process exits the system.
   - Controls the degree of multiprogramming (number of processes in memory).

2. **Short-Term Scheduler (CPU Scheduler):**
   - Selects which process from the ready queue gets CPU time next.
   - Runs frequently for immediate decisions.
   - Must be fast since it directly affects CPU efficiency.

3. **Medium-Term Scheduler:**
   - Manages processes that are partially in memory.
   - Temporarily removes processes from memory (swapping) to improve performance.

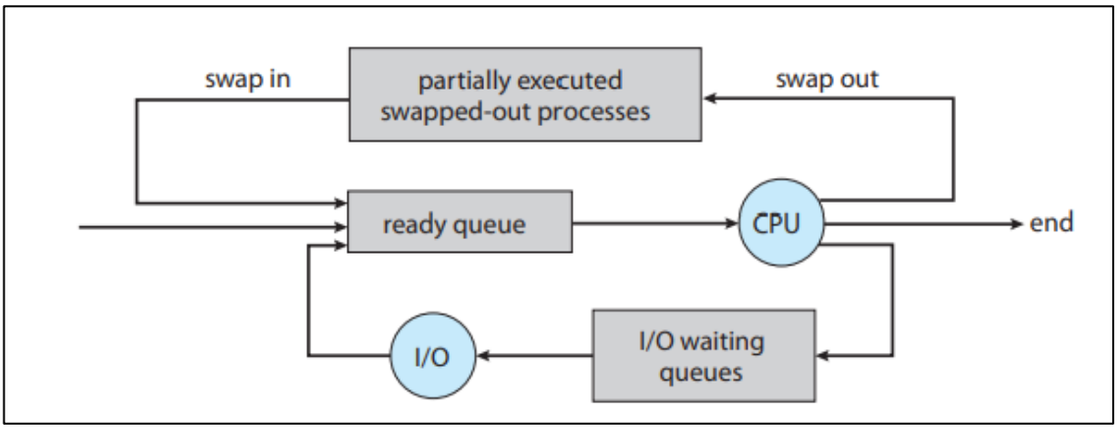



*Figure 8 Addition of medium-term scheduling to the queuing diagram*

**Balancing Process Types**

The long-term scheduler ensures a balanced mix of:

- I/O-bound processes: Spend more time on I/O operations than computations.
- CPU-bound processes: Spend more time on computations with fewer I/O requests.

A balanced mix prevents system inefficiency, where:

- Too many I/O-bound processes leave the CPU idle.
- Too many CPU-bound processes leave devices unused.

**Key Differences:**

| Long-Term Scheduler | Short-Term Scheduler | Medium-term Scheduler |
|---|---|---|
| • Manages the degree of multiprogramming. | • Manages CPU allocation frequently. | • Manages memory load through swapping. |

Some systems, like UNIX and Windows, may lack a long-term scheduler and place all processes directly into memory, relying on user behavior to balance system load.




Here are some useful YouTube videos about process scheduling:

1. multiprogramming and time-sharing, process schedulers, scheduling queues, job queue, and ready queue. Watch here
2. Discusses process scheduling, categories of scheduling, process scheduling queues, and comparisons among schedulers Watch here
3. provides an in-depth look at process scheduling, suitable for undergraduate computer science and IT students Watch here
4. fundamentals of process scheduling in operating systems. Watch here
5. Introduction to Process Scheduling Watch here

# ADDITIONAL NOTE

### *MULTIPROCESS ARCHITECTURE—CHROME BROWSER*

Many websites contain active content such as JavaScript, Flash, and HTML5 to provide a rich and dynamic web-browsing experience. Unfortunately, these web applications may also contain software bugs, which can result in sluggish response times and can even cause the web browser to crash. This isn't a big problem in a web browser that displays content from only one website. But most contemporary web browsers provide tabbed browsing, which allows a single instance of a web browser application to open several websites at the same time, with each site in a separate tab. To switch between the different sites , a user need only click on the appropriate tab. This arrangement is illustrated below:




A problem with this approach is that if a web application in any tab crashes, the entire process—including all other tabs displaying additional websites —crashes as well.

Google's Chrome web browser was designed to address this issue by using a multiprocess architecture. Chrome identifies three different types of processes: browser, renderers, and plug-ins.

- The **browser** process is responsible for managing the user interface as well as disk and network I/O. A new browser process is created when Chrome is started. Only one browser process is created.
- **Renderer** processes contain logic for rendering web pages. Thus, they contain the logic for handling HTML, Javascript, images, and so forth. As a general rule, a new renderer process is created for each website opened in a new tab, and so several renderer processes may be active at the same time.
- A **plug-in** process is created for each type of plug-in (such as Flash or QuickTime) in use. Plug-in processes contain the code for the plug-in as well as additional code that enables the plug-in to communicate with associated renderer processes and the browser process.

The advantage of the multiprocess approach is that websites run in isolation from one another. If one website crashes, only its renderer process is affected; all other processes remain unharmed. Furthermore, renderer processes run in a **sandbox**, which means that access to disk and network I/O is restricted, minimizing the effects of any security exploits.

Source: Operating System Concept, Page 123, Author Abraham Silberschatz

## 2.2 TYPES OF SCHEDULLING

Types of scheduling algorithms play a critical role in managing CPU time and ensuring efficient task execution in an operating system. The main goal of these algorithms is to determine the order in which processes will access the CPU. Below is a detailed explanation of four fundamental scheduling algorithms: First-Come, First-Served (FCFS), Shortest Job Next (SJN/SJF), Round Robin (RR), and Priority Scheduling.

### First-Come, First-Serve (FCFS)

The First-Come, First-Served (FCFS) scheduling algorithm processes tasks in the order they arrive in the ready queue. It is a non-preemptive scheduling algorithm, meaning once a process starts, it runs to completion without being interrupted.

**Working Mechanism:**

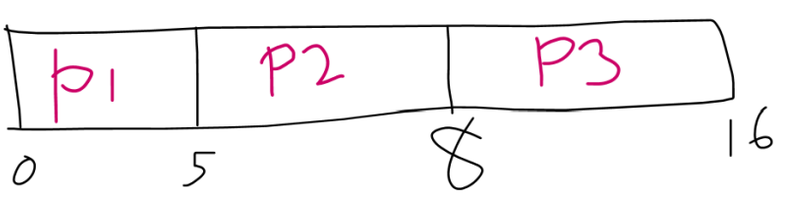
- The CPU is allocated to the first process in the ready queue.
- Each process completes before the next one begins execution.

**Example:**

Processes arrive as follows:

- **P1**: Burst Time = 5 units
- **P2**: Burst Time = 3 units
- **P3**: Burst Time = 8 units

Order of execution: **P1 → P2 → P3**




Calculate the average waiting time = (P1 + P2 + P3) / 3 = 5 + (5+3) + (8+8) = 29/3 = 9.667

Use Case:

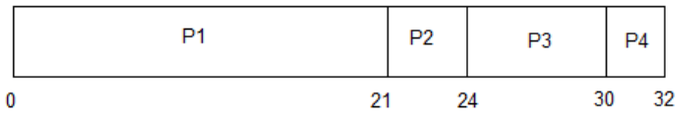
- FCFS is commonly used in batch processing systems where tasks are executed in a sequential order without priority considerations.

## EXAMPLE

Consider the processes P1, P2, P3, P4 given in the below table, arrives for execution in the same order, with Arrival Time 0, and given Burst Time, let's find the average waiting time using the FCFS scheduling algorithm.

| PROCESS | BURST TIME |
|---|---|
| P1 | 21 |
| P2 | 3 |
| P3 | 6 |
| P4 | 2 |

The average waiting time will be = ( 0 + 21 + 24 + 30 )/4 = 18.75 ms




This is the GANTT chart for the above processes

Source:https://www.studytonight.com/operating-system/first-come-first-serve

The average waiting time will be 18.75 ms

- For the above given proccesses, first P1 will be provided with the CPU resources,
  - Hence, waiting time for P1 will be 0
  - P1 requires 21 ms for completion, hence waiting time for P2 will be 21 ms
  - Similarly, waiting time for process P3 will be execution time of P1 + execution time for P2, which will be (21 + 3) ms = 24 ms.
  - For process P4 it will be the sum of execution times of P1, P2 and P3.

## Shortest Job Next (SJN) / Shortest Job First (SJF)

Shortest Job Next (SJN), also called Shortest Job First (SJF), schedules processes based on their burst time, prioritizing the shortest job available.

Working Mechanism:

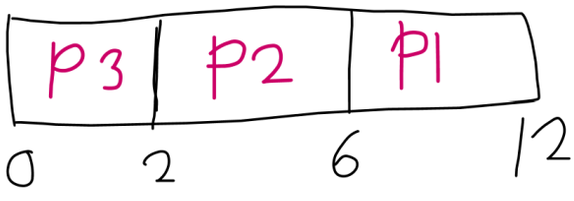
- The process with the shortest burst time is selected first.
- Non-preemptive or preemptive variations exist.
- Non-preemptive: Once a process starts, it cannot be interrupted.
- Preemptive (Shortest Remaining Time First - SRTF): If a new process with a shorter burst time arrives, it can interrupt the current process.

Example:

Processes with the following burst times:

- **P1:** Burst Time = 6
- **P2:** Burst Time = 4
- **P3:** Burst Time = 2

Order of execution: **P3 → P2 → P1**




**Calculation:**

| Process | Arrival Time | Burst Time | Completion Time | Turnaround Time | Waiting Time |
|---|---|---|---|---|---|
| **P3** | 2 | 2 | 4 | 4 - 2 = 2 | 2 - 2 = 0 |
| **P2** | 1 | 4 | 8 | 8 - 1 = 7 | 7 - 4 = 3 |
| **P1** | 0 | 6 | 14 | 14 - 0 = 14 | 14 - 6 = 8 |

**Formula used:**

- **Completion Time (CT):** Time at which the process completes its execution.
- **Turnaround Time (TAT):** `Completion Time - Arrival Time`
- **Waiting Time (WT):** `Turnaround Time - Burst Time`

Use Case:

- SJF is suitable for systems where job lengths are known in advance, such as batch processing and scientific simulations.

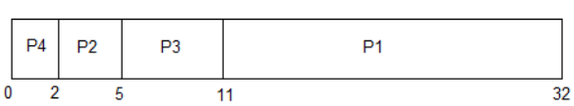
## EXAMPLE

Consider the below processes available in the ready queue for execution, with arrival time as 0 for all and given burst times.

| PROCESS | BURST TIME |
|---|---|
| P1 | 21 |
| P2 | 3 |
| P3 | 6 |
| P4 | 2 |

In Shortest Job First Scheduling, the shortest Process is executed first.

Hence the GANTT chart will be following :




Now, the average waiting time will be = ( 0 + 2 + 5 + 11)/4 = 4.5 ms

As you can see in the GANTT chart above, the process P4 will be picked up first as it has the shortest burst time, then P2, followed by P3 and at last P1.

Source: https://www.studytonight.com/operating-system/shortest-job-first

| | Process | Burst Time | Waiting Time | Turnaround Time |
|---|---|---|---|---|
| 0 | 4 | 2 | 0 | 2 |
| 1 | 2 | 3 | 2 | 5 |
| 2 | 3 | 6 | 5 | 11 |
| 3 | 1 | 21 | 11 | 32 |
| 4 | Average | | **4.5** | **12.5** |

Average waiting time = 4.5

Average turn around time = 12.5

**Formula used:**

- **Completion Time (CT):** Time at which the process completes its execution.
- **Turnaround Time (TAT):** `Completion Time - Arrival Time`
- **Waiting Time (WT):** `Turnaround Time - Burst Time`

## Round Robin (RR)

**Round Robin (RR)** is a preemptive scheduling algorithm where each process is assigned a fixed time slice called a **time quantum**. After the time quantum expires, the process is moved to the back of the ready queue if it is not completed.

**Working Mechanism:**

- Each process gets a time slice (e.g., 4 milliseconds).
- If a process completes within its time slice, it terminates.
- If a process doesn't finish, it is preempted and placed at the end of the queue.

**Example:**

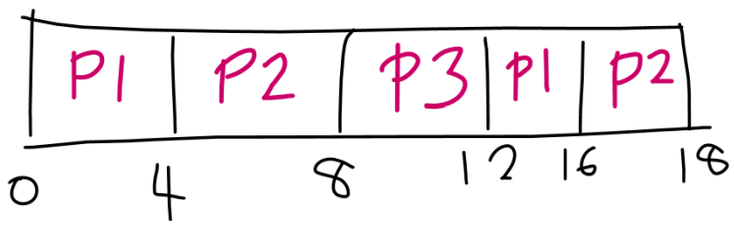
Processes with burst times and a time quantum of 4 units:

- **P1:** Burst Time = 8
- **P2:** Burst Time = 6
- **P3:** Burst Time = 4

Execution order:

**P1 (4 units) → P2 (4 units) → P3 (4 units - completes) → P1 (4 units) → P2 (2 units - completes)**

1. **P1** starts and runs for 4 units. (Remaining burst time = 4)
2. **P2** starts and runs for 4 units. (Remaining burst time = 2)
3. **P3** starts and completes as it requires 4 units.
4. **P1** resumes and completes its remaining 4 units.
5. **P2** resumes and completes its remaining 2 units.




**Calculation:**

| Process | Arrival Time | Burst Time | Completion Time | Turnaround Time | Waiting Time |
|---|---|---|---|---|---|
| **P1** | 0 | 8 | 16 | 16 - 0 = 16 | 16 - 8 = 8 |
| **P2** | 1 | 6 | 18 | 18 - 1 = 17 | 17 - 6 = 11 |
| **P3** | 2 | 4 | 12 | 12 - 2 = 10 | 10 - 4 = 6 |

Formula used:

- **Completion Time (CT):** Time at which the process completes its execution.
- **Turnaround Time (TAT):** `Completion Time - Arrival Time`
- **Waiting Time (WT):** `Turnaround Time - Burst Time`

Use Case:

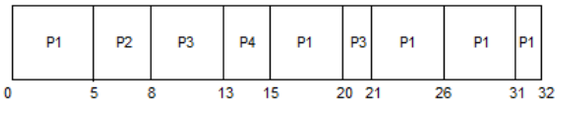
- RR is commonly used in time-sharing systems where multiple users need fair CPU time, such as multitasking operating systems (Windows, Linux).

## EXAMPLE

Let us now cover an example for the same:

| PROCESS | BURST TIME |
|---|---|
| P1 | 21 |
| P2 | 3 |
| P3 | 6 |
| P4 | 2 |

The GANTT chart for round robin scheduling will be,




The average waiting time will be, 11 ms.

In the above diagram, arrival time is not mentioned so it is taken as 0 for all processes.

Note: If arrival time is not given for any problem statement then it is taken as 0 for all processes; if it is given then the problem can be solved accordingly.

The value of time quantum in the above example is 5. Let us now calculate the Turn around time and waiting time for the above example:

| Processes | Burst Time | Turn Around Time | Waiting Time |
|---|---|---|---|
| P1 | 21 | 32 - 0 = 32 | 32 - 21 = 11 |
| P2 | 3 | 8 - 0 = 8 | 8 - 3 = 5 |
| P3 | 6 | 21 - 0 = 21 | 21 - 6 = 15 |
| P4 | 2 | 15 - 0 = 15 | 15 - 2 = 13 |

- Average waiting time is calculated by adding the waiting time of all processes and then dividing by the number of processes.
- Average waiting time formula: Average waiting time = Sum of waiting times of all processes / Number of processes
    - Calculation: 11 + 5 + 15 + 13 = 44
    - Average Waiting Time = 44 / 4 = 11 ms

## Priority Scheduling

Priority Scheduling assigns a priority level to each process, and the CPU is allocated to the highest priority process. Priorities can be assigned based on factors like process importance, memory requirements, or execution deadlines.

**Working Mechanism:**

- Processes are scheduled based on priority values.
- Lower numerical values often indicate higher priority.
- Can be preemptive or non-preemptive.

**Example:**
Consider the following processes with their burst time and priority levels:

| Process | Arrival Time | Burst Time | Priority |
|---|---|---|---|
| P1 | 0 | 8 | 2 |
| P2 | 1 | 4 | 1 |
| P3 | 2 | 2 | 3 |

Execution Order: The priority order (smallest priority executes first): **P2 → P1 → P3**

Calculation:

| Process | Arrival Time | Burst Time | Priority | Completion Time | Turnaround Time | Waiting Time |
|---|---|---|---|---|---|---|
| **P1** | 0 | 8 | 2 | 13 | 13 - 0 = 13 | 13 - 8 = 5 |
| **P2** | 1 | 4 | 1 | 5 | 5 - 1 = 4 | 4 - 4 = 0 |
| **P3** | 2 | 2 | 3 | 15 | 15 - 2 = 13 | 13 - 2 = 11 |

Formula used:

- **Completion Time (CT):** Time at which the process completes its execution.
- **Turnaround Time (TAT):** `Completion Time - Arrival Time`
- **Waiting Time (WT):** `Turnaround Time - Burst Time`

**Use Case:** Priority Scheduling is commonly used in **real-time systems** and **mission-critical applications** where tasks have different importance levels.

## EXAMPLE

Consider following five processes P1 to P5. Each process has its unique priority, burst time, and arrival time.

| Process | Priority | Burst Time | Arrival Time |
|---|---|---|---|
| P1 | 1 | 4 | 0 |
| P2 | 2 | 3 | 0 |
| P3 | 1 | 7 | 6 |
| P4 | 3 | 4 | 11 |
| P5 | 2 | 2 | 12 |

The following step-by-step calculation of Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for Priority Scheduling.

| Process | Arrival Time | Burst Time | Priority | Completion Time (CT) | Turnaround Time (TAT) | Waiting Time (WT) |
|---|---|---|---|---|---|---|
| P1 | 0 | 4 | 1 | 4 ms<br>CT = Arrival Time + Burst Time<br>= 0 + 4 = 4 ms | 4 ms<br>TAT = Completion Time - Arrival Time<br>= 4 - 0 = 4 ms | 0 ms<br>WT = Turnaround Time - Burst Time<br>= 4 - 4 = 0 ms |
| P2 | 0 | 3 | 2 | 7 ms<br>CT = Previous Completion Time + Burst Time<br>= 4 + 3 = 7 ms | 7 ms<br>TAT = Completion Time - Arrival Time<br>= 7 - 0 = 7 ms | 4 ms<br>WT = Turnaround Time - Burst Time<br>= 7 - 3 = 4 ms |
| P3 | 6 | 7 | 1 | 14 ms<br>CT = Previous Completion Time + Burst Time<br>= 7 + 7 = 14 ms | 8 ms<br>TAT = Completion Time - Arrival Time<br>= 14 - 6 = 8 ms | 1 ms<br>WT = Turnaround Time - Burst Time<br>= 8 - 7 = 1 ms |
| P4 | 11 | 4 | 3 | 18 ms<br>CT = Previous Completion Time + Burst Time<br>= 14 + 4 = 18 ms | 7 ms<br>TAT = Completion Time - Arrival Time<br>= 18 - 11 = 7 ms | 3 ms<br>WT = Turnaround Time - Burst Time<br>= 7 - 4 = 3 ms |
| P5 | 12 | 2 | 2 | 20 ms<br>CT = Previous Completion Time + Burst Time<br>= 18 + 2 = 20 ms | 8 ms<br>TAT = Completion Time - Arrival Time<br>= 20 - 12 = 8 ms | 6 ms<br>WT = Turnaround Time - Burst Time<br>= 8 - 2 = 6 ms |

**Final Result:**

| Process | Priority | Burst Time | Arrival Time | Completion Time | Turnaround Time | Waiting Time |
|---|---|---|---|---|---|---|
| P1 | 1 | 4 | 0 | 4 | 4 | 0 |
| P2 | 2 | 3 | 0 | 7 | 7 | 4 |
| P3 | 1 | 7 | 6 | 14 | 8 | 1 |
| P4 | 3 | 4 | 11 | 18 | 7 | 3 |
| P5 | 2 | 2 | 12 | 20 | 8 | 6 |

Average Waiting Time: 2.80 ms

Average Turnaround Time: 6.80 ms

EXERCISE

Answer ALL the questions.

**Exercise 1: First-Come, First-Served (FCFS)**

Given the following processes, calculate Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT):

| **Process** | **Arrival Time** | **Burst Time** |
|---|---|---|
| P1 | 0 | 5 |
| P2 | 1 | 3 |
| P3 | 2 | 6 |

**Tasks:**

1. Draw the Gantt chart.
2. Calculate Completion Time, Turnaround Time, and Waiting Time.

# EXERCISE

## Exercise 2: Shortest Job Next (SJN)

Given the following processes, calculate Completion Time, Turnaround Time, and Waiting Time using SJN scheduling:

| Process | Arrival Time | Burst Time |
|---|---|---|
| P1 | 0 | 4 |
| P2 | 1 | 3 |
| P3 | 2 | 5 |

**Tasks:**

1. Draw the Gantt chart.
2. Calculate Completion Time, Turnaround Time, and Waiting Time.

## Exercise 3: Round Robin (RR)

Given the following processes and a time quantum of 4 units, calculate Completion Time, Turnaround Time, and Waiting Time:

| Process | Arrival Time | Burst Time |
|---|---|---|
| P1 | 0 | 8 |
| P2 | 1 | 6 |
| P3 | 2 | 4 |

**Tasks:**

1. Draw the Gantt chart.
2. Calculate Completion Time, Turnaround Time, and Waiting Time.

## Exercise 4: Priority Scheduling

Given the following processes and priority values (1 is the highest priority), calculate Completion Time, Turnaround Time, and Waiting Time:

| Process | Arrival Time | Burst Time | Priority |
|---|---|---|---|
| P1 | 0 | 7 | 3 |
| P2 | 1 | 4 | 1 |
| P3 | 2 | 5 | 2 |

**Tasks:**

1. Draw the Gantt chart.
2. Calculate Completion Time, Turnaround Time, and Waiting Time.

## 2.3 DEADLOCK MANAGEMENT

Deadlock is a critical issue in process scheduling where a set of processes get stuck indefinitely, each waiting for a resource held by another process. This results in a state where no process can proceed with its execution, leading to system stagnation and resource wastage. Deadlock management is crucial in modern operating systems to ensure smooth multitasking and efficient use of system resources.

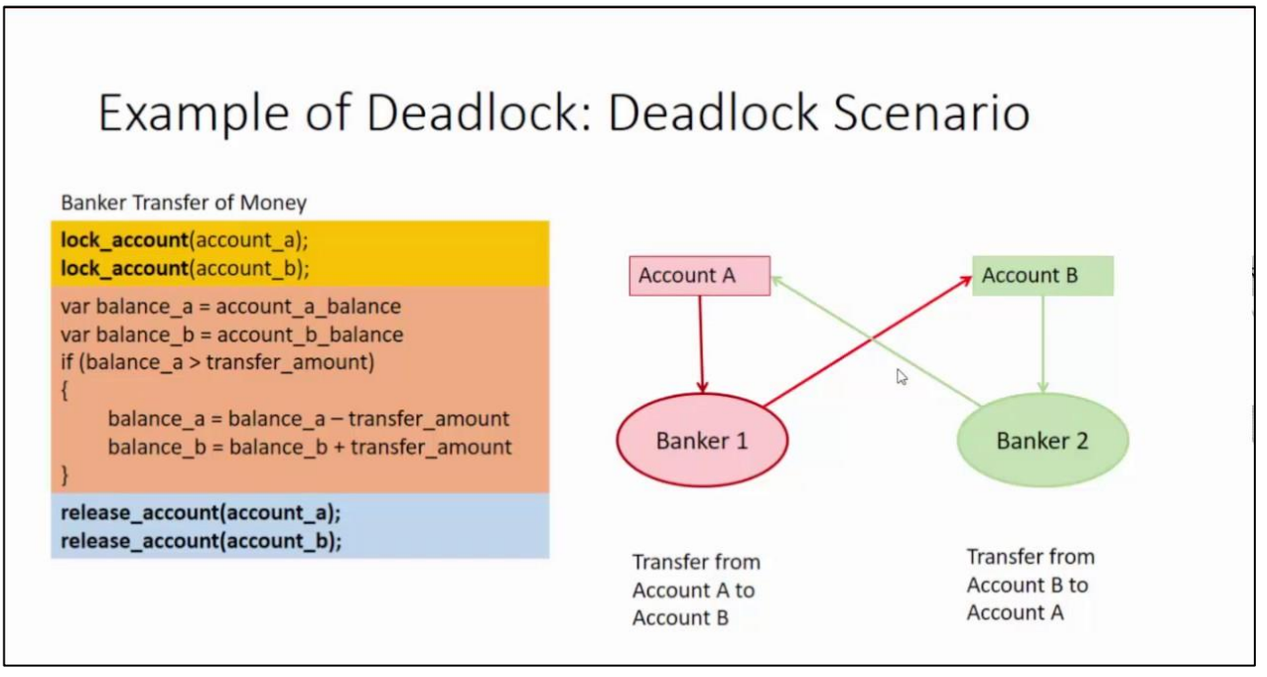



*Figure 9 Deadlock Scenario*

### What is Deadlock?

A deadlock occurs when a group of processes is unable to proceed because each process is holding a resource and waiting for another resource that is already held by another process.

Example of Deadlock:

- Process P1 holds Resource R1 and waits for Resource R2.
- Process P2 holds Resource R2 and waits for Resource R1.
- Since both processes are waiting for each other to release the resource, neither can proceed, causing a deadlock.

## Conditions for Deadlock: (Coffman's Conditions)

A deadlock can occur if the following four conditions hold simultaneously:

1. **Mutual Exclusion:**
   - At least one resource must be held in a non-shareable mode (only one process can use it at a time).
2. **Hold and Wait:**
   - A process holding a resource can request additional resources while holding the currently allocated ones.
3. **No Preemption:**
   - Resources cannot be forcibly taken away from a process; they must be released voluntarily.
4. **Circular Wait:**
   - A closed chain of processes exists where each process waits for a resource held by the next process in the chain.

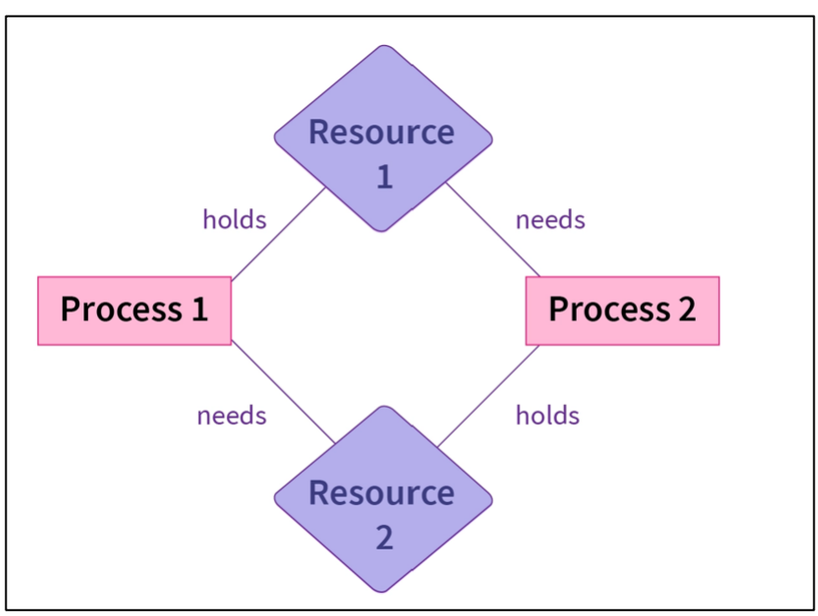



*Figure 10 Deadlock Process*

Deadlock can be managed using four primary strategies: Prevention, Avoidance, Detection and Recovery, and Ignoring Deadlock.

**Deadlock Prevention Techniques:**

- Avoid circular wait conditions.
- Use resource ordering.
- Allow resource preemption.

Example: Two processes holding a printer and a scanner, each waiting for the other.

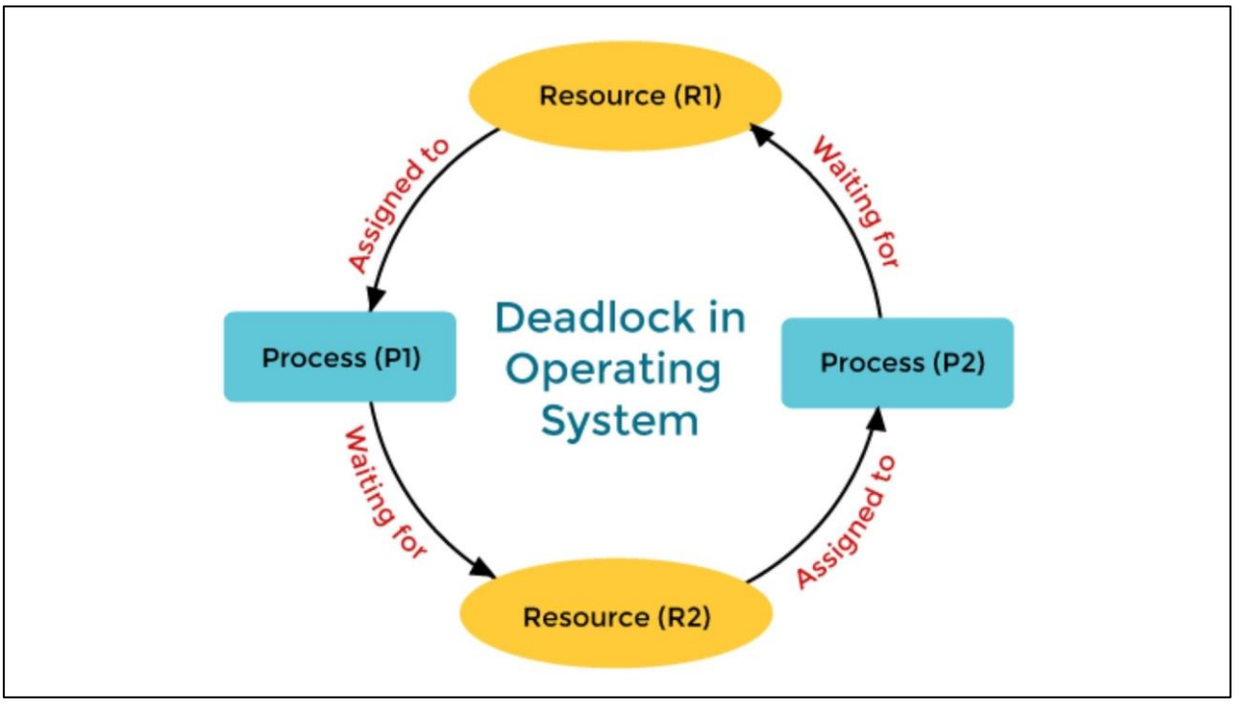



*Figure 11 Two processes holding a printer and a scanner, each waiting for the other.*

**1. Deadlock Prevention:**

**Approach:** Prevent at least one of the Coffman conditions from occurring.

- Breaking Mutual Exclusion: Allow multiple processes to access resources simultaneously (not always practical).
- Breaking Hold and Wait: Require processes to request all needed resources at once before execution starts.
- Breaking No Preemption: Forcefully take resources from processes if needed.
- Breaking Circular Wait: Assign a global ordering of resources and require processes to request resources in increasing order only.

Example: Requiring all processes to request all resources before execution prevents hold and wait.

**2. Deadlock Avoidance:**

**Approach**: Carefully allocate resources to avoid unsafe states where deadlock could occur.

- Banker's Algorithm (for multiple resources):
- Each process declares the maximum number of resources it will need.
- The system only allocates resources if the resulting state is safe.
- Safe State: A state where the system can allocate resources in a way that all processes can complete without deadlock.
- Unsafe State: A state where deadlock may occur but is not guaranteed.

Example:

If process P1 declares it needs 4 units of a resource but currently holds 2, and only 2 units are available, allocation would be denied since granting the request would leave the system in an unsafe state.

**3. Deadlock Detection and Recovery:**

**Approach**: Detect deadlock after it occurs and recover from it.

Deadlock Detection:

- The OS monitors resource allocation and process states.
- Wait-for Graph: A graph representation where a cycle indicates deadlock.

Recovery Methods:

- Process Termination: Terminate one or more processes to break the cycle.
- Resource Preemption: Forcefully take resources from processes.
- Rollback: Roll back processes to a safe state using checkpoints.

**4. Ignoring Deadlock (Ostrich Algorithm):**

**Approach:** Ignore the possibility of deadlock, assuming it is rare.

- Often used in simpler systems where deadlock occurrence is minimal (e.g., single-user systems).

## Deadlock Management Example using Resource Allocation Graph (RAG)

A **Resource Allocation Graph (RAG)** is a graphical representation used for detecting deadlock.

- **Processes:** Represented by circles.
- **Resources:** Represented by squares.
- **Edges:**
  - **Request Edge:** Process → Resource (indicating a request).
  - **Assignment Edge:** Resource → Process (indicating allocation).

**Example:**

1. **P1 → R1** (P1 requests R1)
2. **R1 → P2** (R1 allocated to P2)
3. **P2 → R2** (P2 requests R2)
4. **R2 → P1** (R2 allocated to P1)

This circular structure indicates **deadlock**.

## Summary of Deadlock Handling Approach

| Method | Approach | Advantages | Disadvantages |
|---|---|---|---|
| **Prevention** | Avoids deadlock by breaking conditions | Guaranteed deadlock-free | Poor resource utilization |
| **Avoidance** | Avoid unsafe states | Efficient resource use | Needs prior resource information |
| **Detection & Recovery** | Detect and resolve deadlock after occurrence | Flexible, less restrictive | Performance overhead |
| **Ignoring Deadlock** | Assume rare occurrence | Simple to implement | Risk of system failure |

Deadlock management is essential for ensuring the smooth operation of an operating system. Effective deadlock handling can prevent system hangs, optimize resource usage, and maintain system reliability. While deadlock prevention and avoidance aim to stop deadlock before it occurs, detection and recovery handle it after the fact. The choice of strategy depends on the system requirements, resource availability, and the criticality of process execution. Understanding these strategies helps design more reliable and efficient operating systems.

## 2.4 STARVATION MANAGEMENT

Starvation is a condition in operating systems where a process waits indefinitely for the CPU or other resources due to the continuous allocation of those resources to other higher-priority processes. It occurs when processes with lower priority are continuously overlooked, preventing them from progressing in the system.

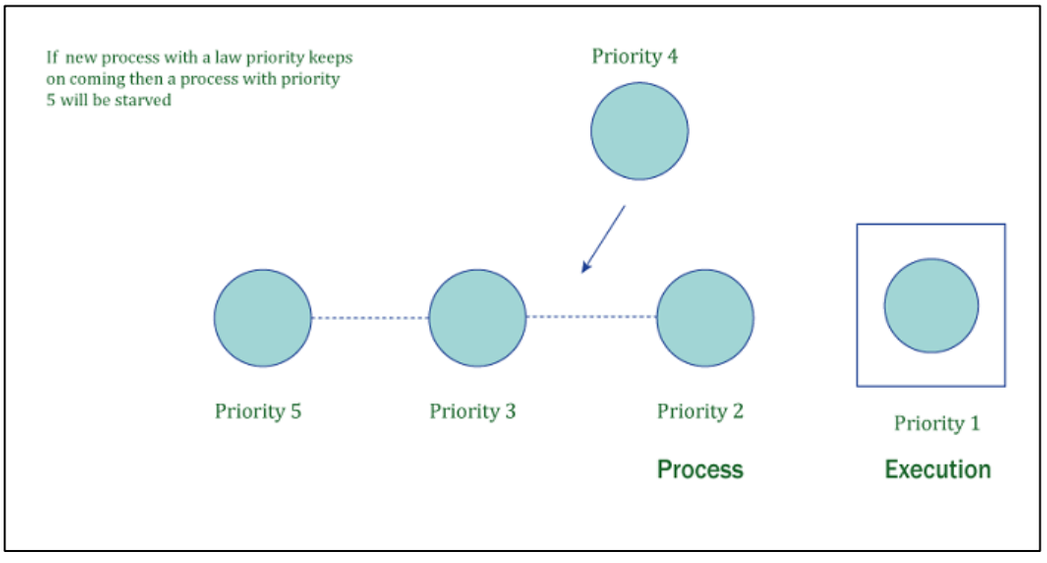



*Figure 12 Starvation Process*

Starvation occurs when a process in a system waits indefinitely for a resource because other processes are constantly given preference. This problem is common in priority-based scheduling algorithms where the CPU keeps allocating resources to higher-priority tasks while lower-priority tasks remain stuck in the waiting queue.

**Example of starvation:**

Consider three processes with the following attributes:

| Process | Arrival Time | Burst Time | Priority |
|---|---|---|---|
| P1 | 0 | 4 | 1 |
| P2 | 1 | 6 | 2 |
| P3 | 2 | 5 | 3 |

- **P1** has the highest priority and keeps getting executed first.
- **P2** with lower priority waits for **P1** to finish.
- **P3** with even lower priority may never get a chance to execute if new high-priority processes keep arriving.

## Causes of Starvation

Starvation mainly arises due to resource management policies and scheduling algorithms. The primary causes include:

**1. Priority-Based Scheduling:**

- Processes with higher priority are scheduled first.

If high-priority tasks keep arriving, low-priority tasks may never get CPU time.

**2. Resource Allocation Policies:**

- When a process requires multiple resources and others with higher priority keep monopolizing them, lower-priority processes get blocked.

**3. Poorly Defined Time Quantum (Round Robin Scheduling):**

- If the time quantum is set too short, some processes may keep preempting others.
- If too long, some processes could dominate CPU usage, causing others to wait indefinitely.

**4. Long Execution Time of Higher-Priority Tasks:**

- Long-running high-priority processes may dominate CPU time, preventing lower-priority tasks from being executed.

## Effects of Starvation

Starvation can lead to several system issues, including:

- **Inefficiency:** Some processes may never complete, leading to wasted system resources.
- **Reduced Fairness:** Processes with lower priority remain neglected.
- **System Imbalance:** Critical tasks may get delayed if the scheduling is unbalanced.
- **Poor User Experience:** In interactive systems, delayed responses affect usability.

## Difference Between Starvation and Deadlock

| Aspect | Starvation | Deadlock |
|---|---|---|
| **Definition** | Process waits indefinitely due to resource preference for others. | Multiple processes wait indefinitely due to circular resource dependency. |
| **Cause** | Priority or unfair resource allocation. | Circular wait condition. |
| **Resolution** | Can be resolved using aging. | Requires deadlock detection and recovery techniques. |
| **Example** | A low-priority process never gets CPU time. | Two processes waiting for each other's resources. |
| **Occurrence** | Common in **priority scheduling**. | Common in **resource allocation systems**. |

## How to Prevent Starvation

Several strategies can be employed to prevent starvation:

**1. Aging Technique:**

- Gradually increases the priority of a process over time.
- Older processes get higher priority to ensure they eventually get executed.
- Example: If a process waits for 5 cycles, its priority can be incremented to prevent indefinite blocking.

**2. Fair Scheduling:**

- Ensures equal CPU time distribution among all processes.
- Round Robin (RR) scheduling ensures each process gets a time slice.

**3. Priority Adjustment:**

- Dynamically adjust process priorities based on waiting time.
- If a process waits too long, the system increases its priority automatically.

**4. Multi-Level Queue Scheduling:**

- Divides the ready queue into multiple levels (e.g., real-time tasks, user tasks).
- Each queue has its own scheduling algorithm, balancing fairness and performance.

**5. Lottery Scheduling:**

- Assigns random tickets to processes and grants CPU access based on a random draw.
- Every process gets a chance proportional to the number of tickets.

Example of preventing starvation using Aging:

Assume the following priority scheduling with aging applied:

| Process | Arrival Time | Burst Time | Initial Priority | Priority After Aging |
|---|---|---|---|---|
| P1 | 0 | 5 | 1 | 1 |
| P2 | 1 | 4 | 3 | 2 |
| P3 | 2 | 3 | 5 | 3 |

- **Initially:** P1 runs first due to its highest priority.
- **With Aging:** Over time, P2 and P3's priority increases, ensuring they execute eventually.

Starvation in process scheduling occurs when lower-priority processes wait indefinitely for resources, leading to inefficiency and unfair resource distribution. While it is common in priority scheduling, it can be effectively managed using strategies such as aging, fair scheduling, and priority adjustment. Implementing proper scheduling policies ensures balanced system performance, efficient resource utilization, and fairness among all processes.

**Case Study: Starvation in Operating Systems**

Scenario:

A company operates a multi-user system where multiple processes run simultaneously. The operating system uses Priority Scheduling to manage tasks. Higher priority tasks (with lower numerical values) are executed before lower priority ones. However, the system has recently been experiencing a problem where some lower-priority tasks are delayed indefinitely, leading to a backlog of unfinished tasks.

Details:

- Process P1 has Priority 1 and requires 5 ms of CPU time.
- Process P2 has Priority 2 and requires 10 ms of CPU time.
- Process P3 has Priority 3 and requires 7 ms of CPU time.
- Process P4 has Priority 5 and requires 2 ms of CPU time.

New high-priority tasks (Priority 1 and 2) keep arriving, preventing the execution of lower-priority processes like P4. As a result, P4 has been waiting for a long time without getting CPU time, while high-priority tasks continue to be executed.

**Questions:**

1. **Identify the Problem:** What scheduling issue is being demonstrated in the above scenario?
2. **Explain the Cause:** Why is process **P4** not getting CPU time?
3. **Effects of Starvation:** What could be the potential effects of starvation on system performance and fairness?
4. **Solution Proposal:** Suggest a solution to prevent starvation in this system.
5. **Role of Aging:** How could **aging** help solve the starvation issue in this scenario?

**Hint:** Consider techniques like **aging**, where a process's priority increases over time, or switching to **Round Robin** scheduling for fairness.




Here are some useful YouTube videos deadlock and starvation.

1. The differences between deadlock and starvation in operating systems Watch here
2. How deadlock occurs when concurrent processes misuse. Watch here
3. Deadlock and Starvation. Watch here
4. Deadlock avoidance strategies in operating systems. Watch here
5. Deadlock and starvation situations in operating system. Watch here

## 2.5 SYNCHRONIZATION IN OS

Synchronization in operating systems is the coordination of multiple processes or threads to ensure their correct and predictable execution. It prevents conflicts when multiple processes access shared resources concurrently, ensuring data consistency and avoiding issues like race conditions, deadlocks, and data corruption.

### What is Synchronization?

**Process Synchronization** is a mechanism used in concurrent programming to ensure that multiple processes can execute without causing data inconsistency or race conditions when accessing shared resources.

**Types of Synchronization:**

There are two primary types of synchronization:

**1. Process Synchronization:**

- Ensures the correct execution of multiple processes running concurrently.
- Prevents issues when processes share resources like CPU time, files, or memory.

**2. Thread Synchronization:**

- Used when multiple threads within a single process need to access shared data.
- Common in multithreaded applications where threads run independently but share resources.

**Why Synchronization is Needed?**

- Prevent **Race Conditions:** Situations where multiple processes access shared data and try to modify it simultaneously, leading to inconsistent results.
- Maintain **Data Integrity:** Ensure data consistency during concurrent execution.
- Avoid **Deadlocks and Starvation:** Proper synchronization prevents both.

**Critical Section Problem**

A **Critical Section** is a section of code where a process accesses shared resources. To ensure consistency, only one process should enter the critical section at a time.

**Conditions for Synchronization (Critical Section Problem Requirements):**

1. **Mutual Exclusion:** Only one process can enter the critical section at a time.
2. **Progress:** If no process is in the critical section, a process must be able to enter it without unnecessary delay.
3. **Bounded Waiting:** Every process should get a chance to enter the critical section within a limited time.

**Synchronization Mechanisms**

Various mechanisms are used in Operating systems to handle synchronization effectively.

1. Semaphores:

- A **Semaphore** is an integer variable used to control access to a resource.
- **Types of Semaphores:**
  - **Binary Semaphore:** Value of 0 or 1 (used for mutual exclusion).
  - **Counting Semaphore:** Value can be greater than 1 (used for multiple resource instances).
- **Operations:**
  - wait() / P(): Decrements the semaphore. If the value becomes negative, the process waits.
  - signal() / V(): Increments the semaphore and releases a waiting process (if any).

**2. Mutex (Mutual Exclusion Object):**

- A Mutex is a locking mechanism where a process must acquire a lock before accessing a shared resource.
- Only one process can hold the lock at a time.
- Operations:
  - Lock: If a mutex is available, a process can lock it and access the resource.
  - Unlock: Releases the lock so another process can access the resource.

**3. Monitors:**

- A Monitor is a higher-level synchronization construct that uses condition variables and locks.
- It encapsulates variables, procedures, and data for synchronized access.

**4. Spinlocks:**

- A Spinlock is a locking mechanism where the process continuously checks for lock availability.
- Used in multiprocessor systems where waiting time is minimal.

**5. Barrier Synchronization:**

- Ensures that all processes reach a certain point before any process continues execution.

**Synchronization Problems and Solutions**

**1. Producer-Consumer Problem (Bounded Buffer Problem):**

- Scenario: A producer generates data and places it in a buffer, while a consumer takes data from the buffer.
- Challenge: Prevent buffer overflow and underflow.
- Solution: Use semaphores to control the buffer size and mutual exclusion.

**2. Readers-Writers Problem:**

- Scenario: Multiple readers can read a shared resource simultaneously, but only one writer can write at a time.
- Challenge: Prevent writers from accessing data while readers are reading.
- Solution: Use semaphores or reader-writer locks.

**3. Dining Philosophers Problem:**

- Scenario: Five philosophers sit at a table with five forks and alternate between thinking and eating.
- Challenge: Prevent deadlock where each philosopher holds one fork and waits for the other.
- Solution: Use semaphores or resource hierarchy to break circular wait conditions.

## Differences Between Deadlocks, Starvation, and Synchronization

| Aspect | Deadlock | Starvation | Synchronization |
|---|---|---|---|
| **Definition** | Processes wait indefinitely for resources. | Lower-priority processes are ignored. | Coordination of processes for shared resources. |
| **Cause** | Circular waiting for resources. | Resource allocation bias. | Simultaneous access to shared data. |
| **Prevention Mechanism** | Breaking circular wait, hold-and-wait. | Aging, fair scheduling. | Mutex, Semaphores, Monitors. |
| **Result** | System halt or freeze. | Long wait times for certain processes. | Consistent data access and execution. |
| **Impact on System** | Complete system lockup. | Degraded system performance. | Improved resource sharing and stability. |
| **Example Scenario** | Two processes holding and waiting for resources. | Backup task delayed due to CPU load. | Two processes accessing a file simultaneously. |

## Conclusion

Synchronization is a critical component of modern operating systems to ensure safe and efficient use of shared resources in concurrent execution. Techniques like semaphores, mutexes, and monitors help in managing process interactions and avoiding issues such as race conditions, deadlocks, and data corruption. Proper synchronization strategies contribute to system reliability, data consistency, and fair resource allocation, which are essential for multitasking and real-time systems.

- **Deadlocks** lead to system freezes due to circular dependencies on resources.
- **Starvation** occurs when lower-priority tasks are indefinitely delayed.
- **Synchronization** ensures controlled access to shared resources, preventing conflicts.

### Best Practices for Handling Concurrency

1. Implement **Proper Locking Mechanisms** (Mutex, Semaphore).
2. Use **Fair Scheduling Algorithms** (Round Robin, Aging).
3. Detect and Prevent **Deadlocks** using avoidance algorithms.
4. Ensure **Process Coordination** using critical section handling techniques.




Here are some useful YouTube videos deadlock and starvation.

1. Synchronization (Operating System) Concept with Example. Watch here
2. Discussing critical sections, race conditions, and various synchronization mechanisms. Watch here
3. Process synchronization, critical sections, race conditions, Peterson's solution, mutex locks, and semaphores. Watch here
4. The Critical-Section Problem. Watch here
5. In-depth look at synchronization in operating systems, suitable for advanced learners seeking a deeper understanding. Watch here

## 2.6 SUMMARY

Process management and scheduling are fundamental concepts in operating systems, ensuring efficient execution of tasks and optimal use of CPU resources. This involves managing processes, their states, control structures, and scheduling algorithms to balance system performance and fairness.

**Processes:** States, Control Blocks, and Context Switching A process is a program in execution, representing the fundamental unit of work in a system. Each process transitions through various states during its lifecycle: new, ready, running, waiting, and terminated. These states help the operating system track whether a process is actively using the CPU, waiting for an event, or has completed its execution.

To manage each process, the Process Control Block (PCB) is used. The PCB stores critical information such as the process ID, current state, program counter, CPU registers, scheduling information, and memory management details. This information ensures that the operating system can manage multiple processes effectively.

Context switching occurs when the CPU switches from executing one process to another. During a context switch, the current state of the running process is saved in its PCB, and the state of the next process to be executed is loaded. This mechanism ensures smooth multitasking but can introduce overhead due to frequent switching.

**Process Scheduling: Algorithms (FCFS, SJF, RR, Priority Scheduling)** Process scheduling determines the order in which processes gain access to the CPU for execution. Several algorithms are used to optimize CPU usage and ensure fairness:

1. **First-Come, First-Served (FCFS):** Processes are executed in the order they arrive. This algorithm is simple but can lead to the convoy effect, where shorter processes wait for longer processes to finish.
2. **Shortest Job First (SJF):** Executes the process with the shortest burst time first, minimizing average waiting time. However, it can lead to starvation where longer tasks are delayed indefinitely.
3. **Round Robin (RR):** Allocates CPU time using a fixed time quantum for each process. It is suitable for time-sharing systems and ensures fairness but may increase context switching overhead.
4. **Priority Scheduling:** Processes are executed based on priority levels. However, starvation can

occur when low-priority tasks are indefinitely delayed due to high-priority tasks dominating CPU time.

Scheduling algorithms are crucial in balancing performance metrics like CPU utilization, throughput, turnaround time, waiting time, and response time.

**Concurrency Issues:** Deadlocks, Starvation, and Synchronization Concurrency arises when multiple processes run simultaneously, often sharing system resources. This can lead to issues like deadlocks and starvation if not properly managed.

1. **Deadlock**: Deadlock occurs when two or more processes are waiting for each other to release resources, leading to indefinite blocking. The four necessary conditions for deadlock are mutual exclusion, hold and wait, no preemption, and circular wait. Deadlocks can be prevented using strategies such as resource allocation graphs or avoided using Banker's Algorithm.
2. **Starvation**: Starvation occurs when lower-priority processes are indefinitely delayed due to continuous execution of higher-priority tasks. Aging can be implemented to prevent starvation by gradually increasing the priority of waiting processes over time.
3. **Synchronization**: Synchronization ensures that multiple processes can run concurrently without leading to race conditions. Tools like semaphores and mutex locks help coordinate process execution and access to shared resources.

Process management and scheduling are critical for the smooth functioning of operating systems, ensuring efficient CPU utilization, fairness, and system stability. By understanding process states, control blocks, and scheduling strategies, along with proper handling of concurrency issues, systems can achieve optimal performance while avoiding deadlocks and starvation.

As conclusion, Process management and scheduling in operating systems involve handling processes from their creation to termination. Processes transition through states such as ready, running, and waiting, managed using a Process Control Block (PCB). Context switching occurs when the CPU shifts between processes, saving and loading process information. Various scheduling algorithms like FCFS, SJF, RR, and Priority Scheduling help determine the order of execution. Issues such as deadlock (when processes block each other while waiting for resources) and starvation (where lower-priority processes are indefinitely delayed) must be managed using techniques like aging and synchronization tools like mutex locks and semaphores. Effective process scheduling ensures balanced CPU usage and fairness in resource allocation.

# Review Question

Here are some True/False questions on Process Management and Scheduling covering the specified topics:

**Processes: States, Control Blocks, Context Switching**

1. **True/False:** A process can be in multiple states simultaneously.
2. **True/False:** The Process Control Block (PCB) contains information about the process state, program counter, and CPU registers.
3. **True/False:** Context switching occurs when the operating system saves the state of a running process and loads the state of the next process.
4. **True/False:** The "terminated" state means a process has completed its execution.
5. **True/False:** The program counter keeps track of the next instruction to be executed in a process.

**Process Scheduling: Algorithms (FCFS, SJF, RR, Priority Scheduling)**

6. **True/False:** In First-Come, First-Served (FCFS) scheduling, processes are executed in the order they arrive.
7. **True/False:** Shortest Job First (SJF) scheduling minimizes the waiting time by executing the shortest burst time process first.
8. **True/False:** Round Robin (RR) scheduling uses a fixed time quantum for process execution.
9. **True/False:** Priority Scheduling always prevents starvation issues.
10. **True/False:** SJF scheduling can be both preemptive and non-preemptive.

**Concurrency Issues: Deadlocks, Starvation, Synchronization**

11. **True/False:** Deadlock occurs when two or more processes are waiting for each other to release resources, causing indefinite blocking.
12. **True/False:** Starvation occurs when a process waits indefinitely due to higher-priority processes continuously getting CPU time.
13. **True/False:** Mutex locks and semaphores are synchronization tools used to avoid race conditions.
14. **True/False:** Deadlock can occur when all four conditions (mutual exclusion, hold and wait, no preemption, and circular wait) hold simultaneously.
15. **True/False:** Aging is a technique used to gradually increase the priority of a process to prevent starvation